

Unidad I

Introducción a algoritmos y programas

Asignatura

Fundamentos de programación

Facilitador

Mgtr. Wilfredo José Meneses Bejarano

2022

I. Introducción

Estimado(a) estudiante, bienvenido a la asignatura Fundamentos de Programación que es la base principal en la formación del Ingeniero en Sistemas de Información, debido a que aporta las habilidades esenciales para el desarrollo de programas de computadora. Estos fundamentos de programación le serán útiles a usted en todas las asignaturas que son parte del área de formación profesional y que están relacionadas con programación de computadoras. La relevancia está en que con los conocimientos que logre adquirir en todo el curso, le permitirá desarrollar algoritmos propios que contribuyan a dar respuestas a problemáticas de su entorno.

Esta asignatura aporta a la construcción del rasgo del perfil profesional “aprovecha e integra las tecnologías de información, software, conforme la misión y visión empresarial”. Se aclara que esta asignatura no es para programar computadoras, sino para crear algoritmos en los que aplicará lógica para obtener soluciones óptimas a los problemas planteados. Si es la primera vez que tiene contacto con este tipo de asignaturas, siéntase tranquilo porque partiremos desde cero y gradualmente iremos adentrándonos en temas avanzados.

En esta primera unidad titulada “Introducción a algoritmos y programas”, abordaremos los conceptos básico relacionados con algoritmos y crearemos nuestras primeras representaciones de soluciones a problemas.

II. Objetivos

- Comprender el concepto de algoritmo y sus distintas interpretaciones.
- Analizar los problemas correctamente determinando los datos de entrada y de salida.
- Expresar algoritmos mediante las distintas técnicas de representación de un algoritmo.
- Desarrollar una conciencia crítica hacia el manejo de la información y la utilización racional de la misma.

III. Cronograma de actividades

Conforme avance en el estudio de este documento encontrará actividades que debe ir completando durante cada semana de clases, para que pueda agendarlos, se los resumimos en la siguiente tabla:

Actividad	Inicio	Fin	Puntaje
Actividad #1: Análisis de videos sobre procesamiento de la información	17/08/2022	18/08/2022	
Actividad #2: Solución de ejercicios	17/08/2022	18/08/2022	
Actividad #3: Lectura recomendada	17/08/2022	18/08/2022	

IV. Desarrollo de contenidos

4.1 Resolución de problemas por ordenador

La resolución de un problema mediante un ordenador consiste en el proceso que **a partir de la descripción de un problema**, expresado habitualmente en lenguaje natural y en términos propios del dominio del problema, **permite desarrollar un programa que resuelva dicho problema**. Es precisamente la solución de problemas la razón de esta asignatura y en esta primera unidad vamos a adentrarnos en los conceptos básicos que servirán de referencia en las próximas unidades.

4.1.1. Introducción al procesamiento de la información

En la asignatura Fundamentos de Tecnologías y Sistemas se habló sobre datos, información y procesos, por lo que ahora vamos a recordarlos a través del análisis de videos, de forma individual daremos respuesta a interrogantes para que luego sean consensuadas de manera colaborativa con el apoyo del docente en el aula de clases.

Actividad #1: Análisis de videos sobre procesamiento de la información

Dado los siguientes videos:

Video 1 (Datos cuantitativos y cualitativos): <https://youtu.be/TNI8tqdh80s>

Video 2 (Procesamiento de datos): <https://youtu.be/Zh8 TC9ukss>

Responda:

1. ¿Qué son datos?
2. De tres ejemplos de datos cuantitativos diferentes a los vistos en el video
3. De tres ejemplos de datos cualitativos diferentes a los vistos en el video
4. ¿Qué es un proceso?
5. ¿Qué es informática?
6. ¿Por qué decimos que hoy los procesos se realizan más rápido?
7. ¿Qué es información?

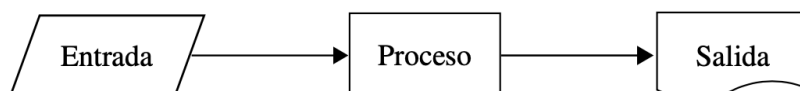
Si ya terminaste la asignación anterior, entonces debes tener muy claros los conceptos y ahora debes asociarlo con la definición de algoritmo que sigue a continuación.

4.1.2. Definición de algoritmo

Un algoritmo es un conjunto detallado y lógico de pasos para alcanzar un objetivo o resolver un problema. Según la RAE (2022), *conjunto ordenado y finito de operaciones que permite hallar la solución de un problema*. Por ejemplo, el instructivo para armar un modelo de avión a escala; cualquier persona, si atiende en forma estricta la secuencia de los pasos, llegará al mismo resultado.

La Figura 1.1 ilustra el funcionamiento conceptual de un algoritmo. A uno o más datos de entrada se les aplica un proceso que involucra una serie de operaciones (pasos), y como resultado se obtienen datos de salida.

Figura 1 – Diagrama conceptual de un algoritmo



Los pasos deben ser suficientemente detallados para que el procesador los entienda. En nuestro ejemplo, el procesador es el cerebro de quien arma el modelo; pero el ser

humano tiende a obviar muchos aspectos y es factible que haga en forma automática algunos de los pasos del instructivo, sin detenerse a pensar en cómo llevarlos a cabo.

Esto sería imposible en una computadora, pues requiere de indicaciones muy puntuales para poder ejecutarlas.

Considérese, por ejemplo, si a una persona se le pide intercambiar los números 24 y 9; con cierta lógica, responderá inmediatamente “9 y 24”.

En tanto, en el procesador de una computadora se tendría que indicar de qué tipo son los datos a utilizar –para este caso, números enteros–; darle nombre a tres variables, digamos, num1, num2 y aux; asignarle a la variable num1 el número 24 y a num2 el 9.

Posteriormente, señalarle que a la variable aux se le asigne el valor contenido en la variable num1; a num1, el contenido en la variable num2; y a esta última, el de la variable aux. Luego, se deben imprimir los valores de las variables num1 y num2 que exhibirán los números 9 y 24.

Se requiere, entonces, una gran cantidad de pasos para indicarle a una computadora que realice la misma tarea que un ser humano.

Otro caso donde podemos notar la forma como el hombre puede obviar pasos es pidiendo a una persona que describa el proceso para preparar agua de limón (limonada). Seguro nos diría que toma una jarra con agua, le pone jugo de limón, azúcar, y listo. Para una computadora lo anterior no tendría sentido, ya que carece de unidades exactas y pasos detallados. Por tanto, los pasos deben tener mayor nivel de precisión, en esta secuencia:

- Inicio del proceso.
- Tomar una jarra de 2 litros de capacidad
- Llenar la jarra a 4/5 partes con agua natural.
- Tomar 4 limones.
- Cortar los limones por la mitad.
- Exprimir los limones dejando caer el jugo sobre el agua en la jarra.
- Tomar el recipiente del azúcar.

- Agregar 4 cucharadas soperas en la jarra con el agua.
- Revolver el agua con el jugo y el azúcar por 2 minutos.
- Fin del proceso.

En conclusión, cuando se elabora un algoritmo, se tomará en cuenta que la computadora es como un niño pequeño al cual se le está enseñando a realizar algo por primera vez; y es necesario concretar lo más posible cada paso que debe dar.

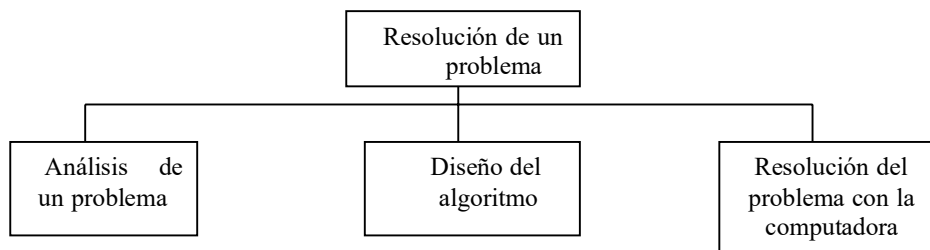
4.1.3. Fases en la resolución de problemas

La resolución de problemas con computadora se puede resolver en tres fases:

- Análisis del problema
- Diseño del algoritmo
- Resolución del algoritmo en la computadora (Implementación y prueba)

En este documento se analizan las tres fases anteriores.

Ilustración 1 - La resolución de un problema



4.1.4. Análisis del problema

El primer paso para encontrar la solución a un problema es el análisis del mismo. **Se debe examinar cuidadosamente el problema a fin de obtener una idea clara sobre lo que se solicita y determinar los datos necesarios para conseguirlo.**

El propósito del análisis de un problema, es ayudar al programador a llegar a una cierta comprensión de la naturaleza del problema. El problema debe estar bien definido si se desea llegar a una solución satisfactoria.

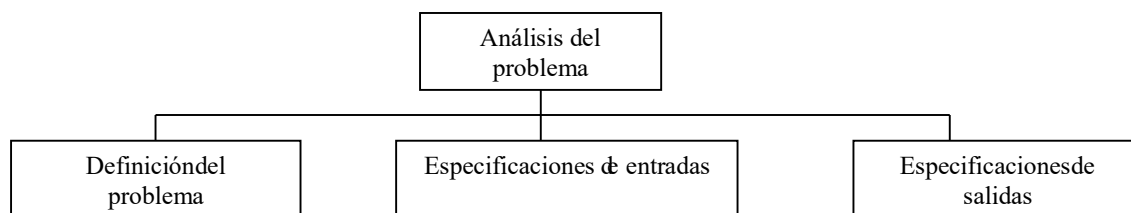
Para poder definir con precisión el problema se requiere que las especificaciones de entrada y salida sean descritas con detalle. Una buena definición del problema, junto con una descripción detallada de las especificaciones de entrada y salida, son los requisitos más importantes para llegar a una solución eficaz.

El análisis del problema exige una lectura previa del problema a fin de obtener una idea general de lo que se solicita. La segunda lectura deberá servir para resolver a las preguntas:

- ¿Qué información debe proporcionar la resolución del problema?
- ¿Qué datos se necesitan para resolver el problema?

La respuesta a la primera pregunta indicará los resultados deseados a las *salidas del problema*. La respuesta a la segunda pregunta indicará que datos se proporcionan a las *entradas del problema*.

Ilustración 2 - Análisis del problema



Ejemplo 1:

Leer el radio de un círculo y calcular e imprimir su superficie y la longitud de la circunferencia.

4.1.5. Análisis

Las entradas de datos en este problema se concentran en el radio del círculo. Dado que el radio puede tomar cualquier valor dentro del rango de los números reales, el tipo de datos radio debe ser real.

Las salidas serán dos variables: superficie y circunferencia, que también serán de tipo real.

Entradas: radio del círculo(variable RADIO).

Salidas: superficie del círculo (variable Área).Circunferencia del círculo (variable Circunferencia).

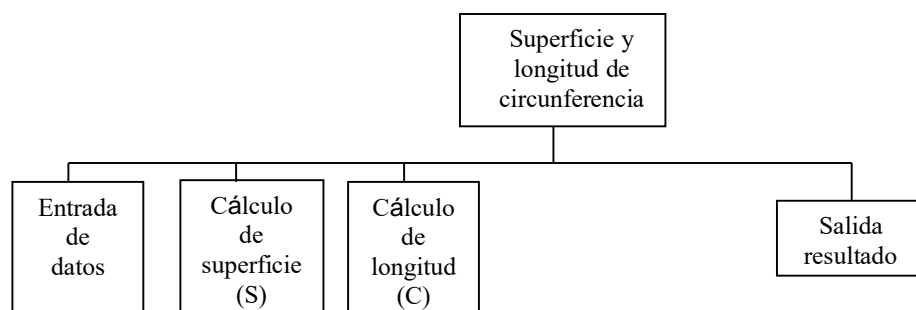
Variables: Radio, Área y circunferencia (tipo real).

4.1.6. Diseño del problema

Una computadora no tiene capacidad para solucionar problemas mas que cuando se le proporcionan los sucesivos pasos a realizar. **Estos pasos sucesivos que indican las instrucciones a ejecutar por la maquina, constituyen, como ya conocemos, el algoritmo.** La información proporcionada al algoritmo, constituye su entrada y la información producida por el algoritmo constituye su salida.

Los problemas complejos se pueden resolver mas eficazmente con la computadora, cuando se rompen en subproblemas que sean más fáciles de solucionar que el original. Este método se suele denominar divide y vencerás (divide and conquer) que consiste en dividir un problema complejo en otros más simples. Así el problema de encontrar la superficie y longitud de un círculo se puede dividir en tres problemas más simples o subproblemas:

Ilustración 3 – Primera descripción del algoritmo



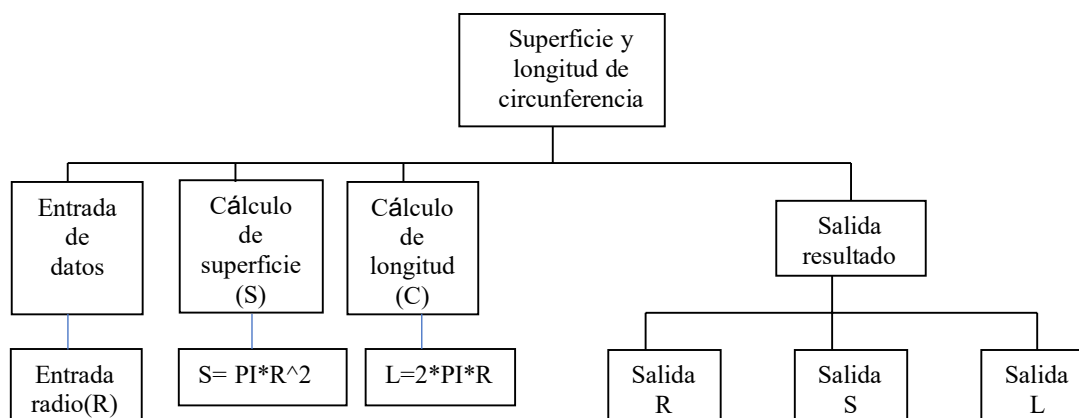
Avancemos con otros conceptos y luego veamos cómo se aplican al problema de la circunferencia y superficie.

La descomposición del problema original en subproblemas más simples y a continuación dividir estos subproblemas en otros más simples que pueden ser implementados para su solución en la computadora se denomina diseño descendente (top-down design). Normalmente los pasos diseñados en el primer esbozo del algoritmo son incompletos e indicaran solo unos pocos pasos (un máximo de doce aproximadamente). Tras esta primera descripción, estos se amplían en una descripción mas detallada con más pasos específicos. Este proceso se denomina refinamiento del algoritmo (stepwise refinement). Para problemas complejos se necesitan con frecuencia diferentes niveles de refinamiento antes de que se pueda obtener un algoritmo claro, preciso y completo.

El problema de cálculo de la circunferencia y superficie de un círculo se puede descomponer en subproblemas más simples: (1) leer datos de entrada, (2) calcular superficie y longitud de circunferencia y (3) escribir resultados (datos de salida).

Subproblema	Refinamiento
<i>leer</i> radio	leer radio
calcular superficie	$\text{superficie} = 3.141592 * \text{radio}^2$
calcular circunferencia	$\text{circunferencia} = 2 * 3.141592 * \text{radio}$
<i>escribir</i> resultados	<i>escribir</i> radio, circunferencia, superficie

Ilustración 4 – Refinamiento de un algoritmo

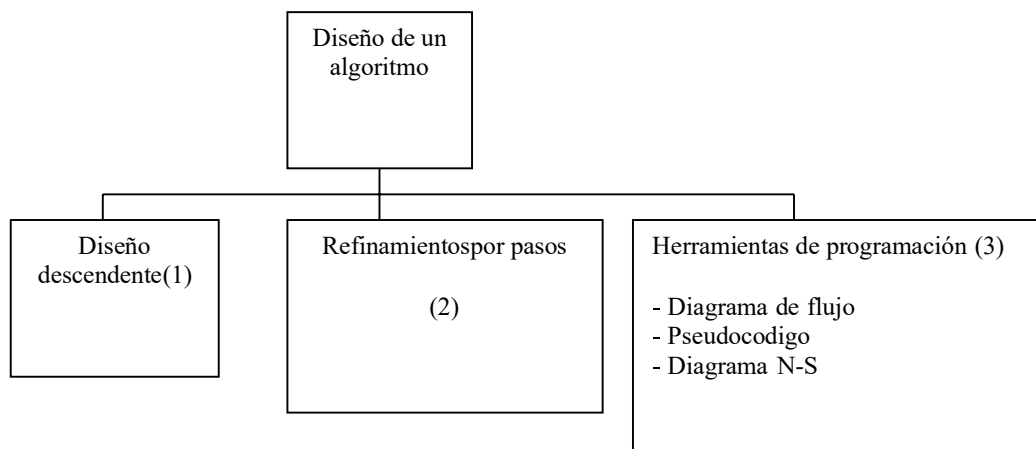


Las ventajas más importantes del diseño descendente son:

- El problema se comprende más fácilmente al dividirse en partes más simples denominadas **módulos**.
- Las modificaciones en los módulos son más fáciles.
- La comprobación del problema se puede verificar fácilmente.

Tras los pasos anteriores (*diseño descendente y refinamiento por pasos*) es preciso representar el algoritmo mediante una determinada herramienta de programación: diagrama de flujo, pseudocódigo o diagrama N.S (Nassi-Shneiderman). Así pues, el diseño de algoritmo se descompone en las fases siguientes:

Ilustración 5 -Fases del Diseño de un algoritmo



4.1.7. Escritura inicial del algoritmo

Como ya se ha comentado anteriormente, el sistema para describir (“escribir”) un algoritmo consiste en realizar una descripción paso a paso con un lenguaje natural del citado algoritmo. Recordemos que un algoritmo es un método o un conjunto de reglas para solucionar un problema. En cálculos elementales estas reglas tienen las siguientes propiedades:

- Deben de estar seguidas de algunas secuencias definidas de pasos hasta que se obtenga un resultado coherente,
- Sólo puede ejecutarse una operación a la vez.

El flujo de control usual de un algoritmo es secuencial; consideremos el algoritmo que responde a la pregunta:

*¿Qué hacer para ver la película **Minions nace un villano**?*

La respuesta es muy sencilla y puede ser descrita en forma de algoritmo, general de modo similar a:

```
Ir al cine

Comprar una entrada (billete o
ticket)

Ver la película

Regresar a casa
```

El algoritmo consta de cuatro acciones básicas, cada una de las cuales debe ser ejecutada antes de realizar la siguiente. En términos de computadora, cada acción se codificará en una o varias sentencias que ejecutan una tarea particular.

El algoritmo descrito es muy sencillo; sin embargo, como ya se ha indicado en párrafos anteriores, el algoritmo general se descompondrá en pasos más simples en un procedimiento denominado *refinamiento sucesivo*, ya que cada acción puede descomponerse a su vez en otras acciones simples.

Así, un primer refinamiento del algoritmo `ir al cine` se puede describir de la forma siguiente:

```
1. Inicio
2. Ver la cartelera de cines por Internet
3. Si no proyectan "Minions nace un villano" entonces
    3.1 decidir otra actividad
    3.2 bifurcar al paso 7
    si no
    3.3 ir al cine.
    Fin_si
4. Si hay cola
    4.1 ponerse en ella
    4.2 mientras haya personas delante hacer
        4.2.1 avanzar en la cola
        fin_mientras
    fin_si
5. Si hay localidades (cupos) entonces
    5.1 comprar una entrada
    5.2 pasar a la sala.
    5.3 localizar la(s) butaca(s)
```

```
5.4 mientras proyectan la película hacer
    5.4.1 ver la película
    fin mientras
5.5 abandonar el cine
si_no
    5.6 refunfuñar
fin_si
6. Volver a casa
7. Fin
```

En el algoritmo anterior existen diferentes aspectos a considerar. En primer lugar, ciertas palabras reservadas se han escrito deliberadamente en negrita (**mientras**, **si no**, etc.). Estas palabras describen las estructuras de control fundamentales y procesos de toma de decisión en el algoritmo. Estas incluyen los conceptos importantes de decisión de selección (expresadas por **si- entonces-si_no** o **if –then-else**) y de repetición (expresadas con **mientras-hacer** o a veces **repetir- hasta e iterar–fin-iterar**, en inglés, while-do y repeat-until) que se encuentra en casi todos los algoritmos, especialmente los de proceso de datos. La capacidad de decisión permite seleccionar alternativas de acciones a seguir o bien la repetición una y otra vez de operaciones básicas

```
Si proyectan la película seleccionada ir al cine
si_no ver la televisión , ir al fútbol o leer el periódico

mientras haya personas en la cola , ir avanzando repetidamente hasta llegar a
la taquilla.
```

Otro aspecto para considerar es el método elegido para describir los algoritmos: empleo de indentación (sangrado o justificación) en escritura de algoritmos. En la actualidad es tan importante la escritura de programa como su posterior lectura. Ello se facilita con la indentación de las acciones interiores a las estructuras fundamentales citadas: selectivas y repetitivas . A lo largo de todo el curso la indentación o sangrado de los algoritmos será norma constante.

Para terminar estas consideraciones iniciales sobre algoritmos, describiremos las acciones necesarias para refinar el algoritmo objeto de nuestro estudio; para ello analicemos la acción.

Localizar las butaca(s)

Si los números de los asientos están impresos en la entrada la acción compuesta se resuelve con el siguiente algoritmo:

```
1.Inicio //algoritmo para encontrar la
butaca del espectador
2.Caminar hasta llegar a la primera fila e
butaca
3.Repetir
    compara numero de fila con numero
    impreso en billete si no son
    iguales, entonces pasar a la
    siguiente fila hasta_que se
    localice la fila correcta
4.Mientras número de butaca no coincida con
número de billete hacer avanzar a
través de la fila a la siguiente
butaca fin_mientras
5.Sentarse en la butaca
6.fin
```

En este algoritmo la repetición se ha mostrado de dos modos, utilizando ambas notaciones, **repetir...hasta_que** y **mientras...fin_mientras**. Se ha considerado también, como ocurre normalmente, que el número del asiento y fila coincide con el número y fila rotulado en el billete.

Actividad #2: Solución de ejercicios

Orientaciones:

1. Se presentan una serie de problemas que deberá resolver de forma individual.
2. Una vez resueltos, puede intercambiar puntos de vistas con sus compañeros de clases.
3. En la próxima sesión el docente seleccionará a las personas que mostrarán su solución compartiendo la solución a través de texto en WhatsApp de grupo.
4. Esta actividad es formativa, pues la idea es que vaya desarrollando habilidades para

A. Diseñar una solución para resolver cada uno de los siguientes problemas y tratar de refinar sus soluciones mediante algoritmos adecuados:

- a) Prepara un café instantáneo.

b) Arreglar un pinchazo de una bicicleta.

c) Freír un huevo.

B. Escribir un algoritmo para:

a) Sumar dos números enteros.

b) Restar dos números enteros.

c) Multiplicar dos números enteros.

d) Dividir un número entero por otro.

4.1.8. Resolución del problema mediante computadora

Una vez que el algoritmo está diseñado y representado gráficamente mediante una herramienta de programación (diagrama de flujo, pseudocódigo o diagrama Nassi-Shneiderman) se debe pasar a la fase de resolución práctica del problema con la computadora.

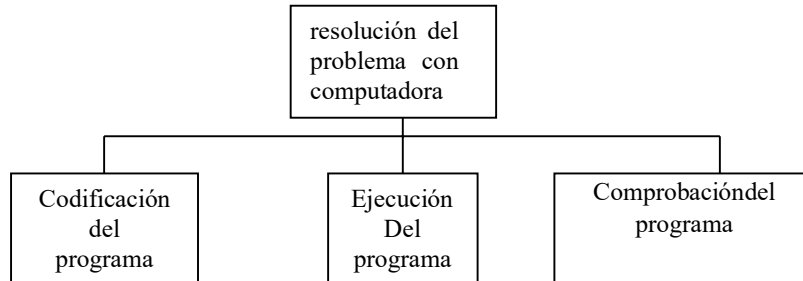
Esta fase se descompone a su vez en las siguientes subfase:

1. Codificación del algoritmo en un programa.
2. Ejecución del programa.
3. Comprobación del programa.

En el diseño del algoritmo éste se describe en una herramienta de programación tal como un diagrama de flujo, diagrama N-S o pseudocódigo. Sin embargo, el programa que implementa el algoritmo debe ser escrito en un lenguaje de programación y siguiendo las reglas gramaticales o sintaxis de este. La fase de conversión del algoritmo en un lenguaje de programación se denomina codificación, ya que el algoritmo escrito en un lenguaje específico de programación se denomina código, aquí se pueden usar lenguajes de programación tales como PHP, Java, JavaScript, C#, por mencionar algunos y dependiendo de lo que se quiera hacer.

Tras la codificación del programa, deberá ejecutarse en una computadora y a continuación de comprobar los resultados pasar a la fase final de documentación.

Ilustración 6 - Resolución del programa mediante computadora



Actividad #3: Lectura recomendada

Orientaciones:

1. Esta asignatura es mayormente práctica, pero requiere de buenas bases teóricas sustentadas en las primeras unidades de estudio, por lo que conforme avance el curso iremos fortaleciendo la cultura de lectura de un libro, en este caso el texto recomendado es Metodología de la Programación, tercera edición de Osvaldo Cairó: <https://tinyurl.com/yrka4y5v>
2. Debe leer, analizar y tomar nota de las páginas 1 – 4.
3. Compruebe la similitud y diferencias entre la propuesta de contenido del libro y la que hemos establecido en este material.
4. La lectura recomendada será consensuada en la próxima sesión de clases a través de preguntas individuales y de grupo.

4.1.9. Características de un algoritmo

Las características fundamentales que debe cumplir todo algoritmo son:

- Un algoritmo debe ser preciso e indicar el orden de realización de cada paso.
- Un algoritmo debe estar definido. Si se sigue un algoritmo dos veces, se debe obtener el mismo resultado cada vez.

- Un algoritmo debe ser finito. Si se sigue un algoritmo, se debe terminar en algún momento; o sea, debe tener un número finito de pasos.

La definición de un algoritmo debe describir tres partes: si tomamos los ejemplos de cocina que ya hemos elaborado, entonces tenemos:

Entrada: ingredientes y utensilios empleados.

Proceso: elaboración de la receta en la cocina.

Salida: terminación del plato (por ejemplo, cordero).

Importante

Aunque la popularización del término Algoritmo ha llegado con el advenimiento de la era informática, algoritmo proviene de Mohammed Al-Khowârizmi, matemático persa que vivió durante el siglo IX y alcanzó gran reputación por el enunciado de las reglas paso a paso para sumar, restar, multiplicar y dividir números decimales; la traducción al latín del apellido en la palabra algorismus derivó posteriormente en algoritmo.

Euclides, el gran matemático griego (del siglo IV antes de Cristo) que inventó un método para encontrar el máximo común divisor de dos números, se considera con Al-Khowârizmi el otro gran padre de la algoritmia (ciencia que trata de los algoritmos).

Por otra parte, debemos estar claros que los algoritmos son independientes tanto del lenguaje de programación en que se expresan como de la computadora que los ejecuta. En cada problema el algoritmo se puede expresar en un lenguaje diferente de programación y ejecutarse en una computadora distinta; sin embargo, el algoritmo será siempre el mismo.

Así, por ejemplo, en una analogía con la vida diaria, una receta de un plato de cocina se puede expresar en español, inglés o francés, pero cualquiera que sea el lenguaje, los pasos para la elaboración del plato se realizarán sin importar el idioma del cocinero.

En la ciencia de la computación y en la programación, los algoritmos son más importantes que los lenguajes de programación o las computadoras.

Un lenguaje de programación es tan sólo un medio para expresar un algoritmo y una computadora es sólo un procesador para ejecutarlo.

Tanto el lenguaje de programación como la computadora son los medios para obtener un fin: conseguir que el algoritmo se ejecute y se efectúe el proceso correspondiente.

Dada la importancia del algoritmo en la ciencia de la computación, un aspecto muy importante será el diseño de algoritmos. A la enseñanza y práctica de esta tarea denominada algoritmia se dedica gran parte de este libro.

4.2 Representación de algoritmos

Las herramientas o técnicas de programación que más se utilizan, que ya se mencionaron y que se emplearán para la representación de algoritmos a lo largo del curso son tres:

1. Pseudocódigo
2. Diagramas de flujo
3. Diagramas Nassi-Schneiderman (N/S). Esta última será una solución alternativa, pero que también suele aplicarse.

Antes de estudiar estas herramientas es necesario tener a mano los siguientes conceptos claves.

4.2.1 Identificadores

Los identificadores son los nombres que se les asignan a los objetos, los cuales se pueden considerar como variables o constantes, éstos intervienen en los procesos que se realizan para la solución de un problema, por consiguiente, es necesario establecer qué características tienen.

Para establecer los nombres de los identificadores se deben respetar ciertas reglas que establecen cada uno de los lenguajes de programación, para el caso que nos ocupa se establecen de forma indistinta según el problema que se esté abordando, sin seguir regla alguna, generalmente se utilizará la letra, o las letras, con la que inicia el nombre de la variable que representa el objeto que se va a identificar.

4.2.2 Constante

Un identificador se clasifica como constante cuando el valor que se le asigna a este identificador no cambia durante la ejecución o proceso de solución del problema. Por ejemplo, en problemas donde se utiliza el valor de PI.

PI = 3.1416.

De igual forma, se puede asignar valores constantes para otros identificadores según las necesidades del algoritmo que se esté diseñando.

4.2.3 Variables

Los identificadores de tipo variable son todos aquellos objetos cuyo valor cambia durante la ejecución o proceso de solución del problema. Por ejemplo, el sueldo, el pago, el descuento, etcétera, que se deben calcular con un algoritmo determinado.

4.2.4 Tipos de variables

Los elementos que cambian durante la solución de un problema se denominan variables, se clasifican dependiendo de lo que deben representar en el algoritmo, por lo cual pueden ser: de tipo entero, real y string o de cadena, sin embargo, existen otros tipos de variables que son permitidos con base en el lenguaje o herramienta de programación que se utilice para crear los programas, por consiguiente, al momento de estudiar algún lenguaje de programación en especial se deben dar a conocer esas clasificaciones. En nuestro caso vamos a utilizar la herramienta PseInt el cual puede descargar e instalar desde la siguiente ruta, según el sistema operativo de su equipo: <http://pseint.sourceforge.net/?page=descargas.php>. PseInt utiliza además de los tipos de antes mencionados, el valor lógico como verdadero o falso.

4.2.5 Literales

Los literales nos permiten representar valores. Los literales son de distintos tipos de datos.

Literales enteros: Ejemplos: 3, 0, -1, ...

Literales reales: Se utiliza el . para separar la parte entera y decimal. Por ejemplo: 3.0, -1.4, 2.345,...

Literales cadenas: Se utiliza las " para indicar una cadena de caracteres, por ejemplo: "Hola", "123", "", ...

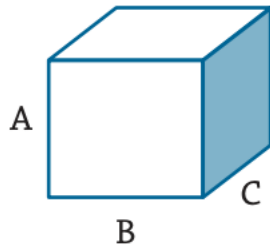
Literales lógicos: sólo pueden tener dos valores: Verdadero y Falso

4.3 Pseudocódigo

Sin duda, en el mundo de la programación el pseudocódigo es una de las herramientas más conocidas para el diseño de solución de problemas por computadora. Esta herramienta permite pasar casi de manera directa la solución del problema a un lenguaje de programación específico. El pseudocódigo es una serie de pasos bien detallados y claros que conducen a la resolución de un problema, nos permite una aproximación del algoritmo al lenguaje natural y por tanto una redacción rápida del mismo.

La facilidad de pasar casi de forma directa el pseudocódigo a la computadora ha dado como resultado que muchos programadores implementen de forma directa los programas en la computadora, cosa que no es muy recomendable, sobre todo cuando no se tiene la suficiente experiencia para tal aventura, pues se podrían tener errores propios de la poca experiencia acumulada con la solución de diferentes problemas.

Por ejemplo, el pseudocódigo para determinar el volumen de una caja de dimensiones A, B y C se puede establecer de la siguiente forma:



1. Inicio.
2. Leer las medidas A, B y C.
3. Realizar el producto de $A * B * C$ y guardarlo en V ($V = A * B * C$).
4. Escribir el resultado V.
5. Fin.

Como se puede ver, se establece de forma precisa la secuencia de los pasos por realizar; además, si se le proporciona siempre los mismos valores a las variables A, B y C, el resultado del volumen será el mismo y, por consiguiente, se cuenta con un final.

4.4 Pseudocódigo en PSeInt

El pseudocódigo detallado anteriormente es de propósito general y es válido, pero en PSeInt existe otra forma de representarlo y para ello vamos primero a ver ciertos aspectos de representación

4.4.1 Estructura del algoritmo

```
Algoritmo circulo
    acción 1;
    acción 2;
    ...
    acción n;
FinAlgoritmo
```

- Comienza con la palabra clave Algoritmo
- Finaliza con la palabra FinAlgoritmo
- La indentación no es significativa, pero se recomienda para que el código sea más legible.
- No se diferencia entre mayúsculas y minúsculas.

4.4.2 Declaración de variables

El perfil "Estricto" nos obliga, al igual que muchos lenguajes de programación, ha indicar explícitamente las variables que vamos a utilizar y sus tipos. Aunque no es necesario es recomendable que las declaraciones se hagan al principio del algoritmo.

Para definir una variable usamos la siguiente instrucción:

```
Definir <var1>, <var2>, ..., <varN> como <Tipo de datos>;
```

Como tipo de datos podemos poner las siguiente opciones:

- Tipo entero: Entero
- Tipo real: Real, Numerico o Numero
- Tipo cadena de caracteres: Caracter, Texto o Cadena
- Tipo lógico: Logico

Por ejemplo:

```
Definir numero1, numero2 como Entero;  
Definir superficie, perimetro como Real;  
Definir nombre como Caracter;  
Definir mayor_edad como Logico;
```

4.4.3 Operadores y expresiones

4.4.3.1 Expresiones

Una expresión es una combinación de variables, literales, operadores, funciones y expresiones, que tras su evaluación o cálculo nos devuelven un valor de un determinado tipo.

Veamos ejemplos de expresiones:

```
a + 7  
(a ^ 2) + b
```

4.4.3.2 Operadores aritméticos

El valor devuelto por una operación aritmética es un número:

+: Suma dos números

-: Resta dos números

*****: Multiplica dos números

/: Divide dos números.

% ó mod: Módulo o resto de la división

^: Potencia

+, **-**: Operadores unarios positivo y negativo

4.4.3.3 Operadores de comparación

El valor devuelto por una operación de comparación es un valor lógico:

>: Mayor que

<: Menor que

=: Igual que

<=: Menor o igual

>=: Mayor o igual

<>: Distinto

La comparación entre cadenas de caracteres se hace según el código ASCII.

4.4.3.4 Operadores lógicos

El valor devuelto por una operación lógica es un valor lógico:

& ó Y: Conjunción, operación AND.

| ó O: Disyunción, operación OR.

~ ó NO: Negación, operación NOT.

Tabla 1 -Tabla de verdad del operador Y

a	b	a Y b
V	V	V
V	F	F
F	V	F
F	F	F

Tabla 2 -Tabla de verdad del operador O

a	b	a O b
V	V	V
V	F	V
F	V	V
F	F	F

Tabla 3 -Tabla de verdad del operador NO

a	No a
V	F
F	V

4.4.3.5 Precedencia de operadores

La precedencia de operadores es la siguiente:

- Los paréntesis rompen la precedencia.
- La potencia
- Operadores unarios
- Multiplicar, dividir y módulo
- Suma y resta
- Operador lógico AND
- Operadores lógico OR
- Operadores de comparación
- Operadores lógicos (not, or, and)

Actividad #4: Solución de ejercicios propuestos

1. Conociendo $A = 5$ y $B = 16$, calcule $(A \wedge 2) > (B * 2)$
2. Conociendo $X = 6$ y $B = 7.8$, calcule $(X * 5 + B \wedge 3 / 4) \leq (X \wedge 3 / B)$
3. Calcule $(15 \geq 7 \wedge 2) \vee (43 - 8 * 2 / 4 <> 3 * 2 / 2)$

4.4.3.6 Asignación de variables

Una vez que hemos definido una variable, podemos asignarle un valor con el operador de asignación (<- ó :=). El dato que se guarda en una variable puede estar expresado por un literal, guardado en otra variable o calculado tras operar una expresión. Por ejemplo:

```
var1 <- 7;  
var2 <- var1;  
var3 <- var1 + var2;
```

Tenemos que tener en cuenta lo siguiente:

- No se puede asignar un valor a una variable que no haya sido definida con anterioridad.
- No se puede utilizar una variable sin inicializar.
- Con cada asignación se pierde el valor anteriormente guardado en la variable.

Las siguientes asignaciones producen error:

- Una variable real (con parte decimal) si se asigna una variable entera.
- Una variable cadena de caracteres si se asigna a una variable numérica.
- Una variable numérica si se asigna a una variable cadena de caracteres.
- Una asignación de una variable lógica a cualquier otra variable de otro tipo, y al contrario.

4.4.3.7 Entrada y salida de información

Entrada de datos

Con la instrucción Leer permite asignar un valor a una (o varias) variables leído por teclado. Hay que tener en cuenta las mismas consideraciones que en las asignaciones.

```
Leer <variable1>, <variable2>, ..., <variableN>;
```


Ejemplo:

```
Definir variable como entero;  
Leer variable;
```

Salida de información

Para mostrar información por pantalla utilizamos la instrucción **Escribir**:

```
Escribir <dato1>, <dato2>, ..., <datoN>;
```

Los datos que mostramos pueden ser literales, variables o expresiones.

También podemos utilizar la instrucción **Escribir Sin Saltar**, para que no se introduzca una nueva línea para cada datos mostrado.

Ejemplo:

```
Escribir "El valor de la variable es ", variable;
```

El pseudocódigo de nuestro ejemplo quedaría de la siguiente forma:

```
Algoritmo VolumenCaja  
    Definir a,b,c, v Como Real;  
    leer a, b, c;  
    v = a * b * c;  
    Escribir "El volumen es: ", v;  
FinVolumenCaja
```

Actividad #5: Lectura recomendada

Orientaciones:

4. Debe leer, analizar, practicar y tomar nota de las páginas 4 – 30.
5. Compruebe la similitud y diferencias entre la propuesta de contenido del libro y la que hemos establecido en este material.
6. Convierta los flujos que se presentan en la lectura a la versión en pseudocódigo con PSeInt.
7. La lectura recomendada será consensuada en la próxima sesión de clases.

4.5 Diagramas de flujo

Los diagramas de flujo son una herramienta que permite representar visualmente qué operaciones se requieren y en qué secuencia se deben efectuar para solucionar un problema dado. Por consiguiente, un diagrama de flujo es la representación gráfica mediante símbolos especiales, de los pasos o procedimientos de manera secuencial y lógica que se deben realizar para solucionar un problema dado.

Los diagramas de flujo facilitan la comunicación entre los programadores y los usuarios, además de que permiten de una manera más rápida detectar los posibles errores de lógica que se presenten al implementar el algoritmo. En la tabla siguiente se muestran algunos de los principales símbolos utilizados para construir un diagrama de flujo, estos pueden variar según la referencia, en nuestro caso estamos usando los de PseInt.

Tabla 4 - Algunos símbolos para diagrama de flujos en PSeInt

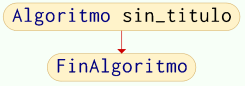
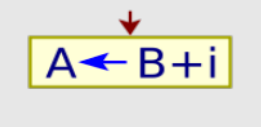
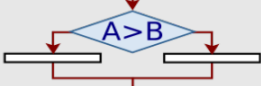
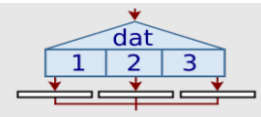
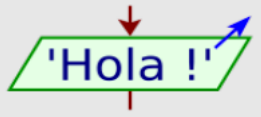
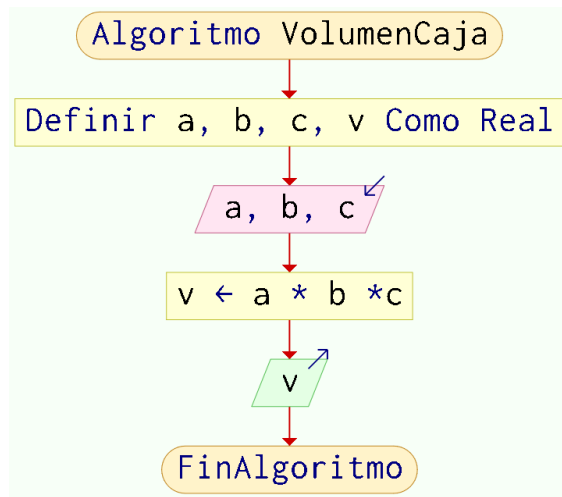
Símbolo	Significado
	Inicia/termina
	Entrada de datos
	Procesos
	Decisión
	Decisión múltiple
	Imprimir resultados

Ilustración 7 – Diagrama de flujo del volumen de una caja



Diagramas Nassi-Schneiderman N/S

El diagrama N-S es una técnica en la cual se combina la descripción textual que se utiliza en el pseudocódigo y la representación gráfica de los diagramas de flujo. Este tipo de técnica se presenta de una manera más compacta que las dos anteriores, contando con un conjunto de símbolos muy limitado para la representación de los pasos que se van a seguir por un algoritmo; por consiguiente, para remediar esta situación, se utilizan expresiones del lenguaje natural, sinónimos de las palabras propias de un lenguaje de programación (leer, hacer, escribir, repetir, etcétera).

Tabla 5 - Principales estructuras utilizadas para construir los diagramas N/S

Símbolo	Tipo de estructura						
<table><tr><td>Acción 1</td></tr><tr><td>Acción 2</td></tr><tr><td>Acción N</td></tr></table>	Acción 1	Acción 2	Acción N	Secuencial			
Acción 1							
Acción 2							
Acción N							
<table><tr><td colspan="2">Expresión lógica</td></tr><tr><td>Sí</td><td>No</td></tr><tr><td>Acción A</td><td>Acción B</td></tr></table>	Expresión lógica		Sí	No	Acción A	Acción B	Selectiva
Expresión lógica							
Sí	No						
Acción A	Acción B						
<table><tr><td>Mientras condición</td></tr><tr><td><table><tr><td>Acciones</td></tr></table></td></tr><tr><td>Fin mientras</td></tr></table>	Mientras condición	<table><tr><td>Acciones</td></tr></table>	Acciones	Fin mientras	De ciclo		
Mientras condición							
<table><tr><td>Acciones</td></tr></table>	Acciones						
Acciones							
Fin mientras							

Ilustración 8 - Diagrama N-S del volumen de una caja

